

V0(초기구축) 코덱스용 작업지시서

1. 작업 목표

아무것도 없는 상태에서 현재 Phoenix detail page 기본 사이트까지 구축한다.

최종 결과는 아래 상태여야 한다.

공개형 상세페이지 생성/편집 웹앱

유저 API 키 입력 방식

브라우저 저장

로컬 다운로드

개별 이미지 편집 안정화

AI 멘트 생성

GitHub/Vercel 배포 가능

현재 레포에서 이 문서를 실행하는 경우에는 누락된 기능을 보완하고, 이미 구현된 기능은 검증한다.

2. V0에서 반드시 포함할 기능

1. 메인 입장 페이지
2. Phoenix 로고와 파비콘 적용
3. /studio 작업 화면
4. API 키 설정 UI
5. 이미지/PDF 업로드
6. 판매 채널 옵션
7. 결과 장수 옵션
8. 출력 비율 옵션
9. 맞춤형 Data 설정 기본 UI
10. OpenAI Image 2.0 생성
11. Google Nano Banana 2 생성
12. 생성 결과 카드
13. 결과 다운로드
14. 결과 저장
15. 브라우저 IndexedDB 최근 작업 저장

16. 섹션별 개별 편집
17. 개별 편집 안정화
18. AI 멘트 생성
19. 빌드 성공

3. 생성할 주요 파일 구조

```
src/app/page.tsx

src/app/studio/page.tsx

src/app/layout.tsx

src/app/globals.css

src/app/api/generate/route.ts

src/app/api/edit-section/route.ts

src/app/api/ai-comment/route.ts

src/app/api/knowledge/route.ts

src/app/api/config/route.ts

src/app/api/client-log/route.ts

src/components/redesign-wizard.tsx

src/components/ui/button.tsx

src/components/ui/card.tsx

src/components/ui/dialog.tsx

src/components/ui/input.tsx

src/components/ui/textarea.tsx

src/components/ui/badge.tsx

src/lib/rag.ts

src/lib/knowledge-access.ts

public/phoenix-ai-logo.png
```

```
public/main-page-image.png
```

```
public/phoenix-web-promo.png
```

4. 구현 단계

4.1 프로젝트 초기화

1. Next.js App Router 프로젝트를 만든다.
2. TypeScript를 사용한다.
3. Tailwind CSS를 사용한다.
4. `lucide-react` 아이콘을 사용한다.
5. 기본 UI 컴포넌트를 만든다.

필수 명령:

```
npm install
```

```
npm run build
```

4.2 메인 페이지 구축

대상 파일:

```
src/app/page.tsx
```

구현 조건:

1. 상단에 Phoenix 로고를 표시한다.
2. 중앙에 입장 버튼을 둔다.
3. 버튼 문구는 `phoenix detail page` 입장을 작게 넣는다.
4. 메인 이미지는 잘리지 않게 표시한다.
5. 하단에는 Phoenix AI Co., Ltd. 푸터를 둔다.
6. 브라우저 탭 파비콘은 Phoenix 로고로 교체한다.

4.3 `/studio` 작업 화면 구축

대상 파일:

```
src/app/studio/page.tsx
```

```
src/components/redesign-wizard.tsx
```

화면 구성:

1. 왼쪽 메뉴

- 홈보드
- 새 작업
- 제작 결과

2. 설정 박스

- OpenAI Image 2.0 설정 상태
- Google Nano Banana 2 설정 상태
- 맞춤형 Data 설정 상태

3. 메인 작업 영역

- 업로드 영역
- 제작 조건
- 요청 문구 입력
- 생성 버튼

4.4 API 키 설정

구현 조건:

1. 사용자가 직접 OpenAI API 키를 입력한다.
2. 사용자가 직접 Google API 키를 입력한다.
3. 키는 브라우저 `localStorage`에만 저장한다.
4. 서버 DB에는 저장하지 않는다.
5. API 키 설정 메뉴에서 입력, 저장, 초기화가 가능해야 한다.

금지:

유저 API 키를 Supabase Database에 저장하지 않는다.

유저 API 키를 로그로 남기지 않는다.

4.5 업로드 기능

지원 대상:

1. 이미지 파일
2. PDF 파일

구현 조건:

1. 이미지 또는 PDF를 업로드하면 생성 요청에 참조 이미지로 사용한다.
2. 긴 PDF는 브라우저에서 이미지로 변환해 전송한다.
3. 업로드 이미지는 생성 요청 전 압축한다.

4.6 생성 API

대상 파일:

```
src/app/api/generate/route.ts
```

구현 조건:

1. OpenAI Image 2.0 생성 지원
2. Google Nano Banana 2 생성 지원
3. 유저가 입력한 API 키를 사용
4. 첫 장 생성 후 나머지 섹션 확장 가능
5. OpenAI high 품질 지연 시 medium 품질 fallback 지원
6. 일부 섹션 실패 시 이미 생성된 섹션은 유지

4.7 결과 화면

구현 조건:

1. 생성된 섹션 이미지를 카드로 표시한다.
2. 각 카드에는 섹션명, 목적, 원본 참조를 표시한다.
3. 각 이미지는 다운로드할 수 있어야 한다.
4. 전체 결과 다운로드 버튼을 제공한다.
5. 결과 저장 버튼을 제공한다.

4.8 브라우저 저장

구현 조건:

1. 저장은 IndexedDB를 사용한다.
2. 저장 대상은 최근 프로젝트다.
3. 홈보드에서 최근 프로젝트를 다시 열 수 있어야 한다.
4. 저장 개수는 과도하게 늘어나지 않도록 제한한다.

금지:

V0에서는 Supabase Database에 프로젝트를 저장하지 않는다.

V0에서는 Supabase Storage에 이미지를 저장하지 않는다.

4.9 섹션별 빠른 편집 메뉴

각 섹션 카드에는 아래 메뉴가 있어야 한다.

1. 카피 강화
2. 디자인 강화
3. CTA 강화
4. 모바일 첫화면
5. 리뷰/UGC 신뢰
6. 불안 제거
7. 중복 레이아웃 줄이기
8. 안전 표현

동작 조건:

1. 메뉴 버튼을 누르면 기본 편집 문구가 입력된다.
2. 기존 입력값이 있으면 아래 줄에 추가한다.
3. 사용자가 직접 문구를 수정할 수 있다.

4.10 AI 멘트 생성

대상 파일:

```
src/app/api/ai-comment/route.ts
```

```
src/components/redesign-wizard.tsx
```

구현 조건:

1. 각 빠른 편집 메뉴 옆에 AI 멘트 생성 버튼을 둔다.

2. 유저가 입력한 OpenAI API 키를 사용한다.
3. 기본 모델은 `gpt-5.4-nano` 를 사용한다.
4. 백업 모델은 `gpt-5.4-mini` 를 사용한다.
5. AI 멘트 생성 요청에는 이미지 `base64`를 보내지 않는다.
6. 섹션 텍스트 정보만 보낸다.

전송 정보:

섹션 ID

섹션명

섹션 목적

원본 참조

기존 프롬프트

선택한 빠른 편집 메뉴

4.11 개별 편집 안정화

대상 파일:

```
src/app/api/edit-section/route.ts
```

```
src/components/redesign-wizard.tsx
```

중요 조건:

개별 편집 요청은 `JSON base64` 방식이 아니라 `FormData` 파일 업로드 방식으로 보낸다.

프론트 구현:

1. 편집 대상 이미지를 브라우저에서 압축한다.
2. 압축된 이미지를 Blob으로 만든다.
3. `FormData` 에 이미지 파일을 넣는다.
4. `model` , `request` , `section` , `project` , `openaiKey` , `googleKey` 를 `FormData` 필드로 넣는다.

5. Content-Type 헤더는 직접 지정하지 않는다.

서버 구현:

1. request.formData() 로 받는다.
2. image 필드의 파일을 Buffer로 변환한다.
3. section, project 는 JSON 문자열로 받아 파싱한다.
4. OpenAI 또는 Google 편집 API로 전달한다.

전송 금지:

```
section.imageUrl
```

```
section.revisions
```

```
프로젝트 전체 이미지 히스토리
```

목표:

```
이미지 데이터가 너무 커서 부분 편집 요청을 보낼 수 없습니다
```

위 오류가 반복되지 않아야 한다.

5. 검증 명령

반드시 실행:

```
npm run build
```

선택 실행:

```
npm run dev:local
```

접속:

```
http://localhost:3002
```


`http://localhost:3002/studio`

6. 성공 기준

1. 메인 페이지가 열린다.
2. `/studio` 가 열린다.
3. API 키 설정이 가능하다.
4. 이미지/PDF 업로드가 가능하다.
5. OpenAI 또는 Google로 1장 생성이 된다.
6. 결과 저장이 된다.
7. 홈보드에서 다시 열 수 있다.
8. 결과 다운로드가 된다.
9. AI 멘트 생성이 된다.
10. 개별 이미지 편집이 된다.
11. 개별 편집 요청 크기 오류가 반복되지 않는다.
12. `npm run build` 가 성공한다.

7. V0에서 하지 않는 것

1. 로그인
2. 유료 유저 구분
3. Phoenix Supabase Storage 저장
4. Phoenix Supabase Database 저장
5. 고급 유저 자기 Supabase 연결
6. 유저 API 키 서버 저장

위 기능은 V1 또는 V2에서 진행한다.